

```

Terminal
[satya:~]$vi bit_field.c
[satya:~]$cc bit_field.c
[satya:~]$./a.out
Hours : -10
Minutes : 12
Seconds : 20
[satya:~]$

```

Figure 1: Actual output

- We will see how 22 is stored in binary form in 5 bits through pictorial representation (Figure 2).
- From the table(as shown in Figure 2), it is very clear that sign bit (b4) is SET, which indicates the value is negative. So, when printed using the *printf* statement, we will get -10 (the decimal value of 10110), because of which we got an unexpected output.

Now that we have understood the problem, how do we fix it? It is very simple; just qualify 'int' to 'unsigned int' just before the hours in the bit field structure, as shown below. The corrected output is shown in Figure 3.

```

#include <stdio.h>
typedef struct bit_field
{
    unsigned int hours : 5;
    unsigned int mins : 6;
    unsigned int secs : 6;
}time_t;
int main()
{
    //Declaration of the variable of type time_t
    time_t alarm;
    //Assigning the values to the different members of
    the bit-field structures
    alarm.hours = 22;
    alarm.mins = 12;
    alarm.secs = 20;
    printf("Hours : %d\nMins : %d\nSecs : %d\n", alarm.
hours, alarm.mins, alarm.secs);
}

```

Bit wise operators definitely provide advantages, but they need to be used a 'bit' carefully. In the embedded programming environment, they might lead to major issues in case they are not handled properly.

Endianness of the architecture and bit fields

In this problem, we will see how Endianness affects the bit fields. Bit fields in C always start at Bit 0, which is the least significant bit (LSB) on Little Endian. But most compilers on Big Endian systems inconveniently consider the most significant bit (MSB)—Bit 0.



Note: Big Endian machines pack bit fields from the most significant byte to the least significant.

Little Endian machines pack bit fields from the least significant byte to the most.

To start with, let us consider the code(Labelled as *byte_order.c*) given below:

```

1 #include <stdio.h>
2 typedef union {
3     unsigned int value;
4     struct {
5         unsigned char one : 8;
6         unsigned char two : 8;
7         unsigned char three : 8;
8         unsigned char four : 8;
9     } bit_field;
10 } data_t;
11
12 int main() {
13
14     data_t var = {0x1A1B1C1D};
15     unsigned char *ptr = (unsigned char *)&var;
16
17     printf("The entire hex value is 0x%X\n", var.
value);
18     printf("The first byte is 0x%X @ %p\n", *(ptr +
0), ptr + 0);
19     printf("The second byte is 0x%X @ %p\n", *(ptr +
1), ptr + 1);
20     printf("The third byte is 0x%X @ %p\n", *(ptr +
2), ptr + 2);
21     printf("The fourth byte is 0x%X @ %p\n", *(ptr +
3), ptr + 3);
22
23     return 0;
24 }

```

b4	b3	b2	b1	b0
1	0	1	1	0

Figure 2: Pictorial representation of the binary value of 22 in 5 bits

```

Terminal
[satya:~]$vi bit_field.c
[satya:~]$cc bit_field.c
[satya:~]$./a.out
Hours : 22
Minutes : 12
Seconds : 20
[satya:~]$

```

Figure 3: Correct output after correct usage of the data type